Quickdress

Milestone 1: Scoping, Requirements and Architecture

Submission document
October 13, 2026

Team
Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

Project links
GitHub: https://github.com/marwanabouelfotuh-gif/Quickdress
Jira: https://swe-quickdress-project.atlassian.net

Contents of this submission

This document brings together everything submitted for Milestone 1. Each part keeps its own section, figure and table numbering, so references such as "Figure 2" or "Table 4" point to the figure or table in the same part.

Deliverable	What it contains	Where
Requirements Specification	System overview, users and stakeholders, product backlog (22 functional, 11 non-functional requirements), scope and sprint plan, use case diagram and descriptions, global and social considerations.	Part A
Design Specification	Component diagram, components and interfaces, sequence diagrams, quality attributes, architectural and design patterns, class diagram, feasibility and risks.	Part B
Presentation	A 17-slide deck for the 20-minute talk, with speaker notes, in Quickdress*Milestone1*Presentation.pptx (and PDF). Part C maps each required topic to its slide.	Separate file and Part C
GitHub repository	Both specifications (Word and PDF), every diagram as an image, and this document, in the "Milestone 1" folder. https://github.com/marwanabouelfotuh-gif/Quickdress	Link
Jira project	Product backlog and sprint board. https://swe-quickdress-project.atlassian.net	Link

Milestone 1 deliverables.

Who prepared what

Document	Prepared by
Requirements Specification (Part A)	Omar Hassan
Design Specification (Part B)	Marwan Abouelfotuh

Contributions by Aya Noaman, Malk Metwally and Nada Salem are tracked on the Jira board.

Part A

Requirements Specification

Quickdress: Requirements Specification

Milestone 1 · Scoping, Requirements and Architecture · October 13, 2026

Prepared by Omar Hassan

Team: Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

Contents

1. System overview
2. Product backlog
3. Scope of the project
4. Use cases
5. Global, cultural, social, environmental, economic, public health, safety and welfare considerations
6. Conclusion

1 System overview

Quickdress is an iOS app that photographs a user's wardrobe once, then recommends a complete outfit from those clothes every morning, matched to the day's weather and calendar.

1.1 What it is

The user captures each clothing item with the phone camera. Quickdress stores the item with its attributes (type, colour, warmth, formality, comfort). Each morning the app presents one outfit for the day; the user can swap a single piece or request a new combination. Everything runs on the device: SwiftUI, SwiftData, and a recommender that starts as a rule-based scorer and can later use an on-device Core ML model. An optional step that asks Google's Gemini model for a suggestion is off by default and runs only if the user turns it on (FR-22).

1.2 Purpose and the problem it solves

Choosing clothes is a daily, repeated decision that costs time and causes decision fatigue. People repeat the same few outfits, forget items they own, and dress for the wrong weather or occasion. Quickdress removes the decision: it turns a 10-minute morning choice into a 10-second confirmation, using only clothes the user already owns.

1.3 Users

User	Need	How Quickdress serves it
University students	Little time in the morning, variable daily schedule (lectures, gym, events)	Calendar-aware outfit each morning
Young professionals	Dress codes that change by meeting; want to avoid repeating outfits in front of colleagues	Formality filter, repeat tracking
People with decision anxiety	Too many choices causes stress	One recommendation, not a catalogue

User	Need	How Quickdress serves it
People unsure about colour matching (including colour-vision deficiency)	Cannot judge which colours go together	Colour matching handled by the recommender

1.4 Stakeholders

Stakeholder	Interest
End users (above)	A fast, trustworthy daily recommendation; privacy of wardrobe photos and calendar
Development team (Aya, Malk, Nada, Marwan, Omar)	Deliverable scope within the course timeline; learning Swift
Course instructor and TAs	Clear requirements, sound architecture, evidence of process (Jira, GitHub)
Apple (platform owner)	App Store guidelines, permission rules for camera, calendar and location
Weather data provider (Apple WeatherKit; Open-Meteo as a backup)	API usage within quota and terms
Google (Firebase AI Logic and the Gemini API)	Only if the opt-in AI step (FR-22) is built: API usage within quota and terms. It receives anonymised item attributes, never photos, calendar text or location.

1.5 Motivation

Existing closet apps (Cladwell, Whering, Acloset) are digital catalogues: they store clothes but still leave the choosing to the user. Quickdress differs in two ways:

- Custom filters that reflect the user's actual day: weather, mode of transport (walking in heat vs. air-conditioned car), comfort level.
- Calendar integration, so a 9 a.m. presentation and a 6 p.m. gym session both shape the recommendation.

The team's background in programming, databases and UI design, plus prior mobile development exposure, makes a native iOS implementation realistic within the course timeline.

2 Product backlog

The backlog lists everything the team has identified, including items that will not fit this term. Priority follows MoSCoW; Section 3 draws the implemented scope from the Must and Should rows.

2.1 Functional requirements

ID	Epic	Requirement	Priority
FR-01	Wardrobe capture	The user shall add a clothing item by taking a photo or choosing one from the photo library.	Must

ID	Epic	Requirement	Priority
FR-02	Wardrobe capture	The system shall store each item with attributes: category (top, bottom, outerwear, shoes, accessory), colour, warmth level, formality level, comfort level, and season.	Must
FR-03	Wardrobe capture	The user shall be able to edit or delete any stored item.	Must
FR-04	Wardrobe browsing	The user shall be able to view the wardrobe as a grid, filtered by category and colour.	Must
FR-05	Daily outfit	The system shall generate one complete outfit (top, bottom, shoes, plus outerwear when needed) each day from the user's own items.	Must
FR-06	Daily outfit	The user shall be able to replace any single piece in the recommended outfit with an alternative suggestion.	Must
FR-07	Daily outfit	The user shall be able to request a completely new outfit.	Must
FR-08	Weather	The system shall fetch the day's forecast for the user's location and use temperature and precipitation when choosing warmth level and outerwear.	Must
FR-09	Daily outfit	The user shall be able to mark an outfit as worn, and the system shall avoid repeating a worn outfit within a configurable number of days.	Should
FR-10	Custom filters	The user shall be able to set daily filters: mode of transport (walking, public transport, car), comfort priority, and formality override.	Should
FR-11	Calendar	The system shall read the day's events from the iOS Calendar (with permission) and raise the outfit's formality when an event is tagged formal, such as a presentation or interview.	Should
FR-12	Saved outfits	The user shall be able to compose and save their own outfits for the recommender to reuse.	Should
FR-13	Wardrobe capture	The system shall suggest the category and dominant colour of a photographed item automatically, which the user may correct.	Should
FR-14	Daily outfit	The system shall send a morning notification at a user-chosen time with the day's outfit.	Could
FR-15	Insights	The system shall show which items have not been worn in the last 30 and 90 days.	Could
FR-16	Daily outfit	The user shall be able to rate a recommended outfit, and the recommender shall learn from ratings.	Could
FR-17	Wardrobe capture	The system shall remove the photo background so items display consistently.	Could
FR-18	Travel	The user shall be able to generate a packing list for a trip of N days at a destination.	Won't (this term)
FR-19	Social	The user shall be able to share an outfit image with friends.	Won't (this term)
FR-20	Laundry	The system shall track items currently in the laundry and exclude them from recommendations.	Won't (this term)

ID	Epic	Requirement	Priority
FR-21	Multi-device	The wardrobe shall sync across the user's devices via iCloud.	Won't (this term)
FR-22	AI suggestions	The user shall be able to turn on AI suggestions. When they are on, the system may ask an external model (Gemini) for an outfit, sending only anonymised item attributes and the day's conditions.	Could

2.2 Non-functional requirements

ID	Quality	Requirement	Priority
NFR-01	Performance	The daily outfit screen shall be visible within 2 seconds of opening the app on an iPhone 12 or newer.	Must
NFR-02	Privacy	Wardrobe photos, item data and calendar information shall be stored and processed on the device only; no personal data leaves the phone without explicit user consent.	Must
NFR-03	Availability (offline)	Wardrobe browsing and outfit generation shall work with no network connection, using the last fetched forecast.	Must
NFR-04	Usability	A new user shall be able to add their first item and receive an outfit within 5 minutes, without instructions.	Must
NFR-05	Performance	Generating an outfit for a wardrobe of up to 200 items shall take under 1 second.	Should
NFR-06	Reliability	The app shall not lose any stored item or photo across app updates or force-quits.	Should
NFR-07	Security	Access to the Camera, Photos, Calendar and Location shall each be requested only when first needed, with an explanation, and the app shall remain usable if any permission is denied.	Should
NFR-08	Maintainability	The recommendation engine shall be behind a single interface so the on-device model can be swapped for an API-based model without changing the UI.	Should
NFR-09	Accessibility	All screens shall support VoiceOver labels and Dynamic Type up to the Accessibility XL size.	Could
NFR-10	Compatibility	The app shall support iOS 17 and later.	Could
NFR-11	Localisation	UI strings shall be externalised so Arabic can be added without code changes.	Could

3 Scope of the project

The team commits to the 11 functional requirements FR-01 to FR-11 and the 8 non-functional requirements NFR-01 to NFR-08. FR-12, FR-13 and FR-22 are stretch goals, built in Sprint 3 only if capacity remains. Everything else in the backlog is deferred.

3.1 Functional requirements in scope

Group	Requirements	Why in scope
Wardrobe capture and browsing	FR-01, FR-02, FR-03, FR-04	Without a stored wardrobe nothing else works; this is the foundation.
Daily outfit	FR-05, FR-06, FR-07	The core promise of the product: open the app, see today's outfit.
Weather	FR-08	The first custom filter and the simplest external integration (WeatherKit is a first-party Apple API).
Context filters	FR-09, FR-10, FR-11	These are the differentiators named in the pitch: no repeats, transport and comfort filters, calendar awareness.
Stretch	FR-12, FR-13, FR-22	Saved outfits, automatic attribute suggestion and the opt-in Gemini step improve the experience, but the app is complete without them. Built only if Sprint 3 has capacity.

3.2 Non-functional requirements in scope

NFR-01 to NFR-08: performance (2 s to first screen, 1 s to generate), on-device privacy, offline operation, 5-minute first-run usability, data durability, permission handling, and a swappable recommendation engine. NFR-09 to NFR-11 (accessibility, iOS version floor, localisation) are followed as good practice but not formally tested this term.

3.3 Out of scope this term

FR-14 to FR-21: notifications, wear insights, rating-based learning, background removal, travel packing, social sharing, laundry tracking and iCloud sync. Each needs either a server, extra ML work, or an additional Apple framework, and none is required to prove the concept.

3.4 Why the scope is achievable

The scope is split into three sprints, one per remaining course milestone, and each sprint ends with a working app that can be demonstrated. The riskiest work (the recommender, Sprint 2) is placed in the middle so Sprint 3 remains as buffer.

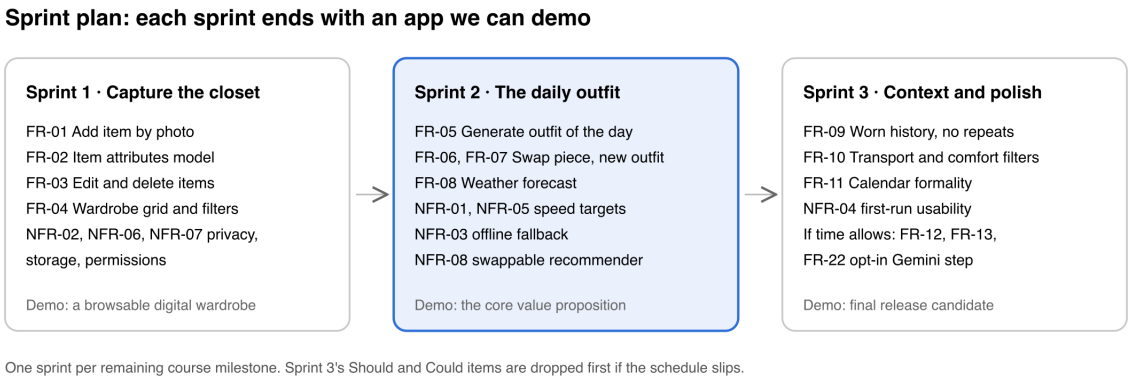


Figure 1. Sprint plan: three sprints, one per remaining milestone.

Five team members give roughly 10 person-weeks per sprint. Sprint 1 is mostly SwiftUI and SwiftData CRUD, well documented and suitable for a team learning Swift. Sprint 2 is the recommender, which in its first version is a rule-based scorer over item attributes (warmth vs.

temperature, formality vs. calendar, colour compatibility table) rather than a trained model; this removes the ML risk from the critical path, and the on-device Core ML version can replace it behind the same interface (NFR-08). Sprint 3 adds filters that are mostly extra scoring rules plus one framework integration (EventKit) and, if capacity remains, the opt-in Gemini step behind the same interface (FR-22).

Open question for the team: confirm the number of sprints and their dates from the course schedule so the plan above can carry real dates.

4 Use cases

Quickdress has one primary actor, the User, and three secondary actors that the system calls: Apple WeatherKit, the iOS Calendar (EventKit) and, only when the user has turned on AI suggestions, Google's Gemini model through Firebase AI Logic. The diagram covers the in-scope requirements FR-01 to FR-11 and the stretch requirement FR-22.

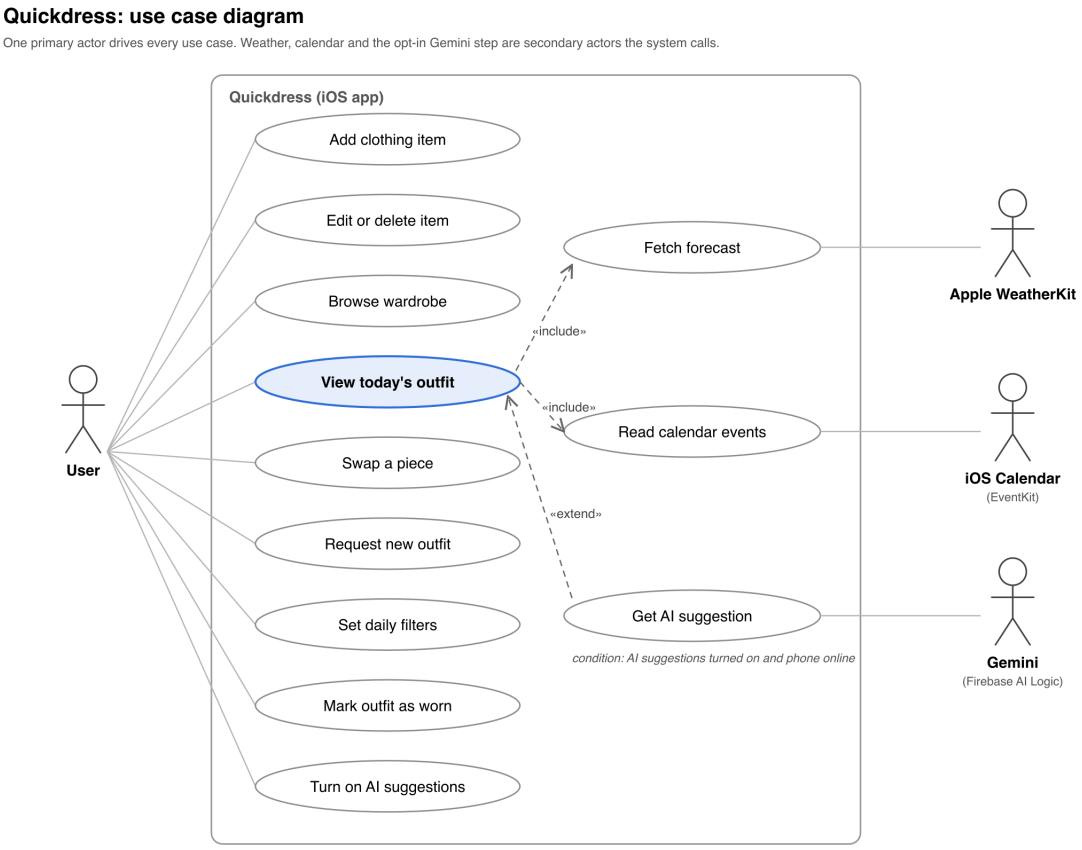


Figure 2. Use case diagram: one primary actor, three secondary actors, twelve use cases.

Viewing today's outfit always includes fetching the forecast and reading calendar events; if either permission is denied the use case still completes with defaults (NFR-07). Getting an AI suggestion extends it only when the user has turned the feature on and the phone is online.

4.1 Use case descriptions

Use case	Actor	Precondition	Main flow	Postcondition	Requirements
Add clothing item	User	Camera or Photos permission granted	1. User taps Add. 2. Takes or picks a photo. 3. Sets category, colour, warmth, formality, comfort, season. 4. Saves.	Item stored locally with photo and attributes	FR-01, FR-02
Edit or delete item	User	Item exists	1. User opens an item. 2. Changes attributes or taps Delete. 3. Confirms.	Item updated or removed; outfits containing it are invalidated	FR-03
Browse wardrobe	User	At least one item	1. User opens Wardrobe. 2. Filters by category or colour. 3. Taps an item for detail.	None	FR-04
View today's outfit	User	At least one top, bottom and pair of shoes stored	1. App opens on Today. 2. System fetches forecast and calendar (included). 3. Recommender scores combinations and shows the best one.	Outfit of the day displayed and cached	FR-05, FR-08, FR-11, NFR-01
Swap a piece	User	Outfit displayed	1. User taps a piece. 2. System offers the next best alternatives for that slot. 3. User picks one.	Outfit updated	FR-06
Request new outfit	User	Outfit displayed	1. User taps New outfit. 2. System generates the next best full combination.	Outfit replaced	FR-07
Set daily filters	User	None	1. User opens Filters. 2. Sets transport mode, comfort priority, formality override. 3. Applies.	Recommendation regenerated with the filters	FR-10
Mark outfit as worn	User	Outfit displayed	1. User taps Wearing this.	Outfit logged with today's date; excluded from recommendations for the configured period	FR-09
Turn on AI suggestions	User	None	1. User opens Settings. 2. Reads what data would be sent. 3. Turns on AI suggestions.	Consent stored; the Gemini step may run when online	FR-22, NFR-02

Use case	Actor	Precondition	Main flow	Postcondition	Requirements
Fetch forecast	System (WeatherKit)	Location permission granted, network available	1. System requests today's hourly forecast. 2. Stores it locally.	Forecast cached; last forecast used when offline	FR-08, NFR-03
Read calendar events	System (EventKit)	Calendar permission granted	1. System reads today's events. 2. Detects formal keywords or user tags.	Formality level for the day set	FR-11
Get AI suggestion	System (Gemini)	AI suggestions on, network available, no answer from the on-device model	1. System sends anonymised item attributes and the day's conditions. 2. Gemini returns one item per slot. 3. System checks the reply against the wardrobe.	Outfit returned; on any failure the rule-based recommender answers instead	FR-22, NFR-02, NFR-03

5 Global, cultural, social, environmental, economic, public health, safety and welfare considerations

The most significant consideration is privacy: Quickdress holds photographs of a person's clothes, their location and their calendar. Processing everything on the device (NFR-02) is a design decision made because of this, not only for performance.

Area	Consideration	How the requirements respond
Privacy and security	Wardrobe photos, calendar entries and location reveal daily routine and whereabouts.	On-device storage and inference (NFR-02); permissions requested one at a time with an explanation and the app usable without them (NFR-07); no account, no server of our own, no analytics. The only external AI call (FR-22) is off by default and sends anonymised attributes only.
Public health and safety	Dressing for the wrong weather causes discomfort and, in Cairo summers, heat-related risk; morning decision stress is a small daily wellbeing cost.	Weather-driven warmth and outerwear choice (FR-08); one recommendation instead of a catalogue (FR-05) to reduce decision fatigue.
Cultural and social	Dress norms differ by religion, workplace and family; modest dress, hijab-compatible combinations and local occasions (Eid, Ramadan iftars, university events) are common for the target users. The recommender must not impose a single Western style norm.	Formality and comfort are user-defined attributes, not inferred from fashion datasets (FR-02); the user can save their own outfits as examples (FR-12); no body-shape or appearance scoring anywhere in the backlog.
Environmental	Fast fashion is driven partly by people feeling they have nothing to wear.	Recommending only from what the user owns, and tracking what has not been worn (FR-09, FR-15), encourages re-wearing instead of buying.

Area	Consideration	How the requirements respond
Economic	Target users are students and early-career professionals with limited budgets.	The app is free, has no in-app purchases, and never links to shopping. Development cost is low: first-party Apple frameworks and no server hosting. WeatherKit needs a paid Apple Developer membership, with free Open-Meteo as a backup, and the opt-in Gemini step uses Google's lowest-cost model.
Global and accessibility	The first release is English-only and iOS-only, which limits reach in Egypt where Android dominates. Colour matching is a real barrier for users with colour-vision deficiency.	Strings externalised for Arabic (NFR-11); VoiceOver and Dynamic Type (NFR-09); colour matching handled by the system so the user never has to judge colours. Android is noted as a future direction, not a requirement.
Public welfare	A recommender that nudges users towards particular clothes could be exploited for advertising.	NFR-02 rules out third-party data sharing without consent, and the scope contains no monetisation feature or commercial recommendations.

6 Conclusion

Quickdress solves a small but daily problem, choosing what to wear, by recommending one outfit from the user's own wardrobe, matched to the weather and the day's calendar. This document fixes the Milestone 1 baseline: a backlog of 22 functional and 11 non-functional requirements, a committed scope of FR-01 to FR-11 and NFR-01 to NFR-08 with FR-12, FR-13 and FR-22 as stretch goals, twelve use cases, and a three-sprint plan in which each sprint ends with a demonstrable app. The accompanying Design Specification shows how the MVVM and layered architecture, with the recommender behind a single interface, meets the non-functional requirements committed here. The next delivery is Sprint 1: a working digital wardrobe with capture, editing and browsing.

Part B

Design Specification

Quickdress: Design Specification

Milestone 1 · Scoping, Requirements and Architecture · October 13, 2026
Prepared by Marwan Abouelfotuh
Team: Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

This document is the companion to the Quickdress Requirements Specification. Requirement IDs (FR-xx, NFR-xx) refer to that document.

Contents

1. High-level architecture
2. Explanation of the architecture
3. Quality attributes
4. Architectural and design patterns
5. Detailed design: class diagram
6. Technical feasibility and risks
7. Conclusion

1 High-level architecture

1.1 Overview

Quickdress is a single iOS app with no backend of its own. Wardrobe data, photos and calendar information stay on the phone. The app makes only two kinds of network call: one to Apple WeatherKit for the forecast and, if the user opts in (FR-22), one to Google's Gemini 3.1 Flash-Lite model through Firebase AI Logic.

The app has three layers, and the presentation layer follows MVVM (Model, View, ViewModel):

- **Presentation layer.** SwiftUI views and their ViewModels. A view draws the screen. Its ViewModel holds the screen's state and calls the layers below.
- **Domain layer.** The app logic in plain Swift. It checks that a clothing item is valid, builds a picture of the day (weather, events, filters) and picks the outfit. It contains no UI code and makes no framework calls.
- **Data and services layer.** A set of protocols, each with an adapter around one Apple framework or external service: SwiftData, the file system, WeatherKit, EventKit, Core Location, UserDefaults and Firebase.

Dependencies point downward only. The layering is relaxed rather than strict, which means a ViewModel may call the data layer directly when there is no logic in between, for example to read the wear log or save filter settings. Sending those calls through a domain class that only forwards them would add code without adding behaviour. Calls never go upward, and no view talks to a framework or the database directly.

1.2 Component diagram

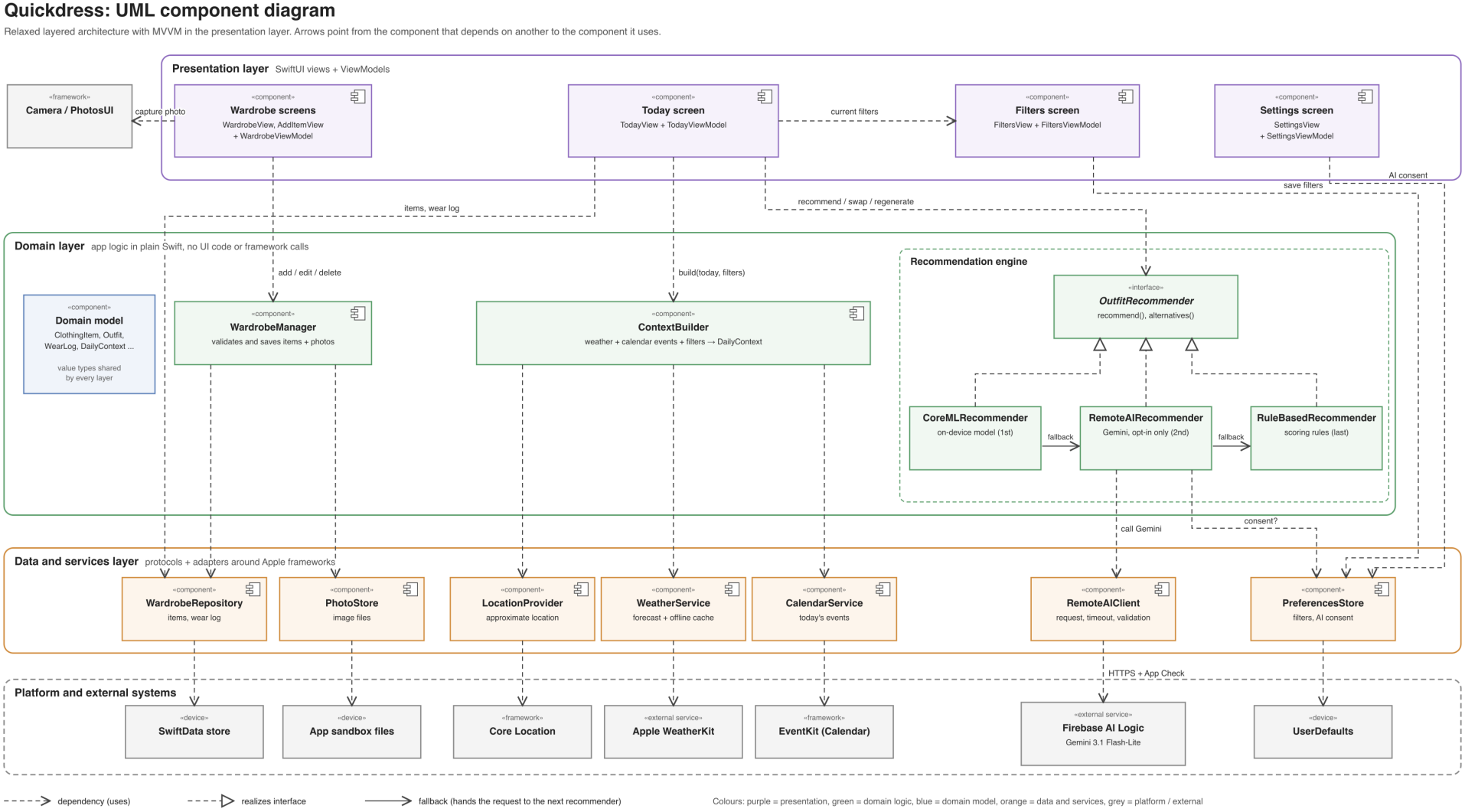


Figure 1. UML component diagram of Quickdress. Colours match the class diagram in Figure 4.

1.3 Components and responsibilities

Table 1 lists every component in Figure 1, what it is responsible for, what it depends on, and which requirements it serves.

Component	Layer	Responsibility	Depends on	Requirements
Today screen TodayView, TodayViewModel	Presentation	Shows the day's outfit. Handles swapping a piece, asking for a new outfit and marking the outfit as worn. Shows a loading state and an offline badge.	ContextBuilder, OutfitRecommender, WardrobeRepository, Filters screen	FR-05, FR-06, FR-07, FR-09, NFR-01
Wardrobe screens WardrobeView, AddItemView, WardrobeViewModel	Presentation	Shows items in a grid with category and colour filters. Adds, edits and deletes items. Opens the camera or the photo library.	WardrobeManager, PhotosUI	FR-01 to FR-04
Filters screen FiltersView, FiltersViewModel	Presentation	Sets transport mode, comfort priority, formality override and the no-repeat period. Hands the current filters to the Today screen.	PreferencesStore	FR-09, FR-10
Settings screen SettingsView, SettingsViewModel	Presentation	Holds the switch for AI suggestions and explains, before the user turns it on, exactly what data is sent.	PreferencesStore	FR-22, NFR-02
WardrobeManager	Domain	Validates item attributes (required fields, ratings from 1 to 5). Saves the photo first and the item second, and deletes both together.	WardrobeRepository, PhotoStore	FR-01 to FR-03, NFR-06
ContextBuilder	Domain	Combines the forecast, calendar events and filters into one DailyContext. Works out the day's formality level. Fills in defaults when a source is missing.	WeatherService, CalendarService, LocationProvider	FR-08, FR-10, FR-11, NFR-03, NFR-07
OutfitRecommender (interface)	Domain	The one contract for producing an outfit: `recommend()` and `alternatives(for:slot:)`.	None	NFR-08
CoreMLRecommender	Domain	Scores combinations with an on-device Core ML model. Passes the request on when no model is bundled or its confidence is low.	Core ML model file, next recommender	FR-05, NFR-02
RemoteAIRecommender	Domain	Asks Gemini for an outfit when the user has opted in and the phone is online. Checks the reply against the wardrobe. Otherwise passes the request on.	RemoteAIClient, PreferencesStore, next recommender	FR-05, FR-22, NFR-02
RuleBasedRecommender	Domain	Scores items with fixed rules: warmth against temperature, formality against the day, a colour compatibility table,	None	FR-05 to FR-10, NFR-03, NFR-05

Component	Layer	Responsibility	Depends on	Requirements
		transport and comfort, and recent wear. Always returns an outfit if the wardrobe has a top, a bottom and shoes.		
Domain model	Domain	Types shared by every layer: ClothingItem, Outfit, WearLog, DailyContext, DailyFilters, Forecast, CalendarEvent and the enumerations.	None	FR-02
WardrobeRepository	Data	Stores and fetches items and the wear log.	SwiftData	FR-02, FR-03, FR-09, NFR-06
PhotoStore	Data	Saves, loads and deletes photo files in the app's sandbox. The database keeps only a reference to each file.	File system	FR-01, NFR-06
WeatherService	Data	Fetches the forecast for a date and place, and caches the last result for offline use.	WeatherKit	FR-08, NFR-03
CalendarService	Data	Asks for calendar access the first time it is needed and reads the day's events.	EventKit	FR-11, NFR-07
LocationProvider	Data	Gets an approximate location for the forecast.	Core Location	FR-08, NFR-07
PreferencesStore	Data	Keeps the saved filters and the AI consent setting.	UserDefaults	FR-09, FR-10, NFR-02
RemoteAIClient	Data	Builds the anonymised request, calls Gemini through Firebase AI Logic with a timeout, and parses the JSON reply.	Firebase AI Logic	FR-22, NFR-02, NFR-08

Table 1. Components, responsibilities and the requirements they serve.

1.4 Relationships and interfaces

The components follow four rules:

1. **Dependencies point down.** A component may use components in its own layer or in a lower one, never in a higher one.
2. **Every service sits behind a protocol.** The domain and presentation layers depend on `WardrobeRepository`, `WeatherService` and so on, never on the concrete adapters (`SwiftDataWardrobeRepository`, `WeatherKitService`, `EventKitCalendarService`). That is why Figure 1 shows only the protocols.
3. **Objects are wired together in one place.** The app's entry point (`QuickdressApp`) creates the adapters, the recommender chain and the managers, then passes them to the ViewModels through their initialisers. Tests and SwiftUI previews pass fakes in the same way.
4. **The domain model is shared.** Items, outfits and the other model types are used by all three layers. They are marked `@Model` so that SwiftData can store them directly. This is the only place where the domain layer knows about a framework. We accept it because a

second set of storage types would double the model code for little gain. All queries and the `ModelContext` stay inside `SwiftDataWardrobeRepository`.

Table 2 lists the interfaces between components.

Interface	Provided by	Used by	Main operations
OutfitRecommender	CoreMLRecommender, RemoteAIRecommender, RuleBasedRecommender	TodayViewModel	`recommend(items, wearLog, context, filters) -> Outfit?` `alternatives(for:slot:) -> [ClothingItem]`
WardrobeRepository	SwiftDataWardrobeRepository	WardrobeManager, TodayViewModel	`fetchItems()`, `save(_:)`, `delete(_:)`, `fetchWearLog(since:)`, `logWorn(_:on:)`
PhotoStore	File-based PhotoStore	WardrobeManager	`save(image) -> ref`, `load(ref)`, `delete(ref)`
WeatherService	WeatherKitService (OpenMeteoWeatherService as a backup, see R1)	ContextBuilder	`forecast(for:at:)`, `cachedForecast()`
CalendarService	EventKitCalendarService	ContextBuilder	`requestAccess()`, `events(on:)`
LocationProvider	CoreLocationProvider	ContextBuilder	`currentLocation() -> Location?`
PreferencesStore	UserDefaultsPreferencesStore	FiltersViewModel, SettingsViewModel, RemoteAIRecommender	`filters`, `aiConsent`
RemoteAIClient	FirebaseGeminiClient	RemoteAIRecommender	`suggestOutfit(request) async throws -> OutfitSuggestion`

Table 2. Interfaces between components.

2 Explanation of the architecture

This section follows two requests through the system, then explains the recommendation engine and the alternatives we rejected. Figure 2, on the next page, shows the first request.

2.1 Viewing today's outfit

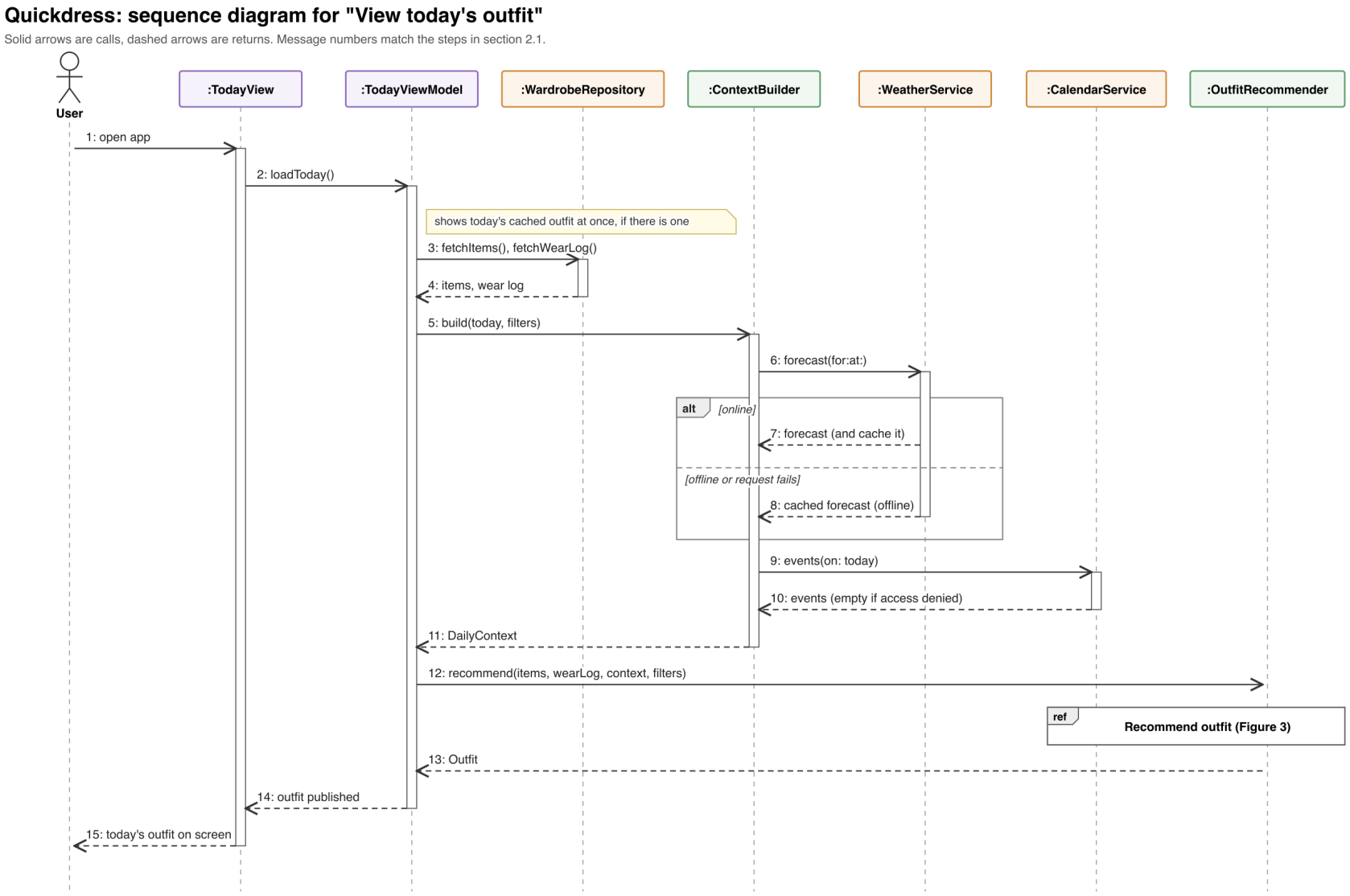


Figure 2. Sequence diagram for the "View today's outfit" use case (FR-05, FR-08, FR-11, NFR-01). The recommender step is shown in Figure 3.

Figure 2 shows the main use case. The numbers in brackets are the message numbers in the figure.

1. The user opens the app and `TodayViewModel.loadToday()` runs (1, 2). If an outfit was already generated today, it is shown from the cache straight away, so the screen is never blank (NFR-01).
2. The ViewModel loads the items and the recent wear log from `WardrobeRepository` (3, 4).
3. It asks `ContextBuilder` for a `DailyContext`, passing today's date and the filters from the Filters screen (5).
4. `ContextBuilder` gets an approximate location from `LocationProvider` and the forecast from `WeatherService` (6, 7). If the network is down or the request fails, `WeatherService` returns the last cached forecast instead and the context is marked offline (8). The Today screen then shows a small badge. If location permission is denied, the context uses a default temperature range and the user is told the outfit ignores the weather.
5. `ContextBuilder` reads today's events from `CalendarService` (9, 10). If calendar access is denied, the list is empty and formality comes only from the filters (NFR-07).
6. It works out the formality level. An event the user tagged as formal, or one whose title contains a keyword such as "presentation" or "interview", raises the level. A formality override set in the filters wins over both. The finished `DailyContext` goes back to the ViewModel (11).
7. The ViewModel calls `recommend()` on the `OutfitRecommender` it was given (12, 13). That object is the first link of the fallback chain, so the ViewModel never knows which recommender produced the outfit. Figure 3 in section 2.3 shows what happens inside the chain.
8. The outfit is published (14). SwiftUI redraws the screen, and the outfit is cached for the rest of the day (15).

Swapping a piece (FR-06) calls `alternatives(for:slot:)` on the same chain and replaces one slot with `Outfit.replacing(slot:with:)`. Asking for a new outfit (FR-07) calls `recommend()` again, excluding the current outfit. Marking an outfit as worn (FR-09) calls `logWorn(_:on:)` on the repository, and every recommender then skips that combination for the no-repeat period.

2.2 Adding a clothing item

The Wardrobe screen opens the camera or the photo library. Permission is asked for only at that moment (NFR-07). The user sets category, colour, warmth, formality, comfort and seasons, then taps Save. `WardrobeManager.addItem` validates the attributes and asks `PhotoStore` to write the image file, which returns a reference. Only then does it save the item, with that reference, through `WardrobeRepository`. If saving the item fails, the manager deletes the photo it has just written, so no orphan files are left behind (NFR-06). Deleting an item runs in the opposite order. If the deleted item is part of today's cached outfit, the outfit is regenerated.

2.3 The recommendation engine and its fallback chain

The engine has three recommenders. All three implement `OutfitRecommender` and are linked in a fixed order. Each one holds a reference to the next (the `fallback` property in the class diagram), and passes the request on when it cannot answer. Figure 3 shows the chain at work.

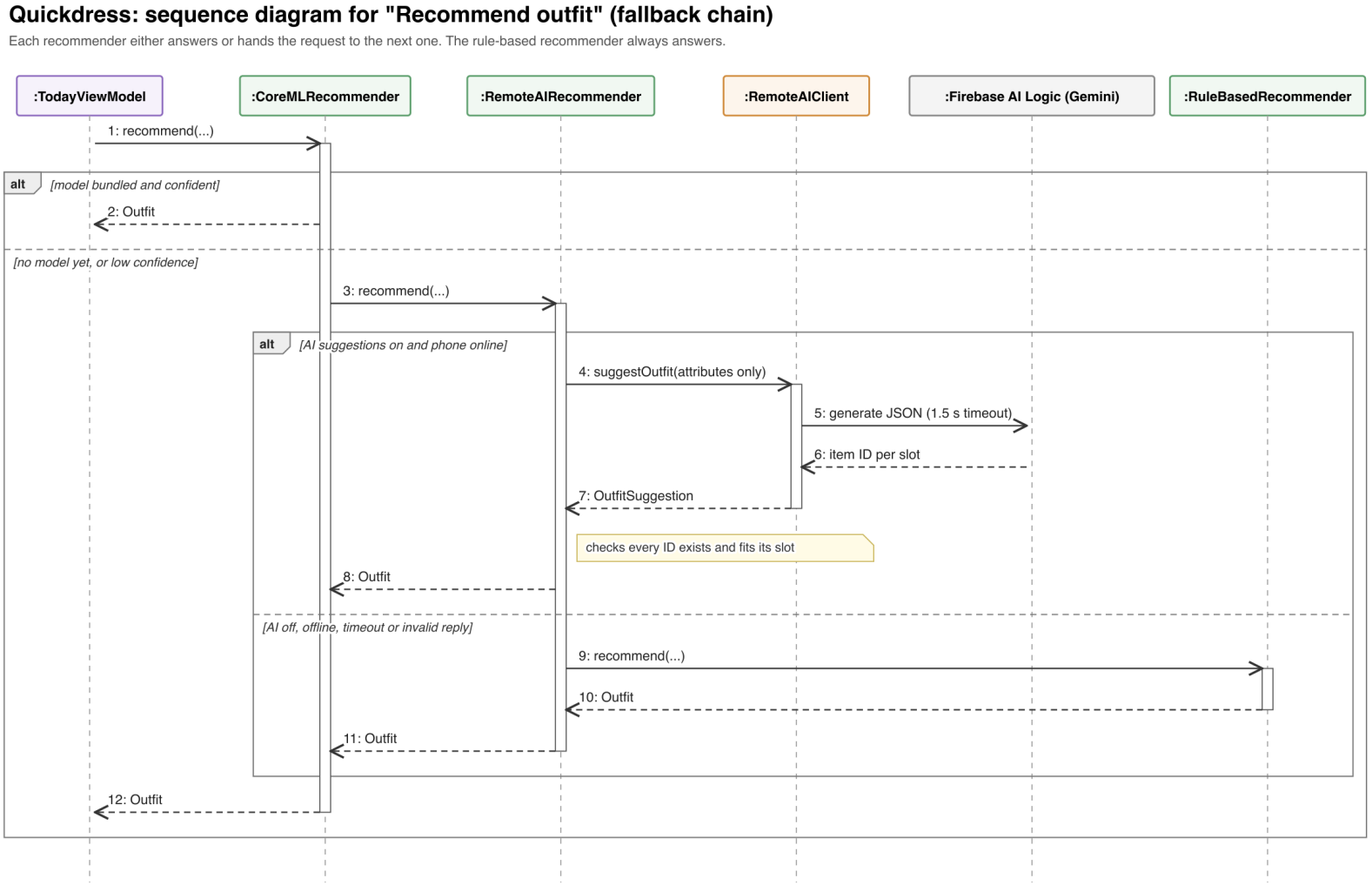


Figure 3. Sequence diagram for "Recommend outfit", the fallback chain referenced in Figure 2 (FR-05, FR-22, NFR-02, NFR-03, NFR-08).

Recommender	Used when	Data leaving the phone	Works offline	Built in
1. CoreMLRecommender	A trained model is bundled with the app and its confidence is above a threshold.	None	Yes	After the rule-based version is stable (stretch goal)
2. RemoteAIRecommender	The Core ML step passed the request on, the user has turned on AI suggestions in Settings, and the phone is online.	Anonymised attributes only (see below)	No	Sprint 3, if there is capacity (FR-22)
3. RuleBasedRecommender	Always, as the last link.	None	Yes	Sprint 2

Table 3. The three recommenders in fallback order.

Until a Core ML model exists, `CoreMLRecommender` forwards every request, so the chain already works in Sprint 2 when only the rule-based scorer is built. The last link needs nothing from outside the phone, which means the app can always produce an outfit. This is what NFR-03 asks for.

The rule-based scorer first scores each item on its own for its slot (warmth, formality, comfort, season, recent wear), keeps the best five per slot, and only then scores the combinations for colour compatibility. That is at most 5 × 5 × 5 × 5 = 625 combinations however large the wardrobe is, which keeps generation well under the one-second limit in NFR-05.

What is sent to Gemini

When the remote step runs, `RemoteAIClient` sends:

- for each item: its ID, category, colour, warmth, formality, comfort and seasons;
- for the day: the temperature range, the chance of rain, the formality level, the transport mode and the comfort priority;
- the IDs of items worn during the no-repeat period.

It never sends photos, item names, calendar event titles or the user's location. Gemini 3.1 Flash-Lite is asked for structured JSON output that names one item ID per slot. `RemoteAIClient` rejects any reply that is not valid JSON, names an ID that is not in the wardrobe, or puts an item in the wrong slot. In any of those cases, or if no reply arrives within 1.5 seconds, the request goes on to the rule-based recommender. The model name is stored in Firebase Remote Config, as Firebase recommends, so it can be changed without an app update.

The AI switch is off by default. Before the user turns it on, the Settings screen explains in one sentence what is sent. This is the explicit consent that NFR-02 requires.

Why Firebase AI Logic instead of calling the Gemini API directly

Calling the Gemini API directly would mean shipping an API key inside the app, and anything inside an app bundle can be extracted. Firebase AI Logic keeps the key on Google's side and uses App Check (App Attest on iOS) to accept requests only from genuine copies of our app. We still run no server of our own and have no user accounts, as the Requirements Specification commits to. Firebase is a Google-hosted service, used only by the opt-in step (FR-22).

2.4 Alternatives considered

Alternative	Why we did not choose it
Model, View, Controller (MVC)	SwiftUI has no view controllers, so in practice the logic ends up inside the views, where it is hard to test. MVVM fits SwiftUI's data binding (`@Observable`) directly.
Client and server with our own backend	It would allow syncing and a server-side model, but it breaks on-device privacy (NFR-02), needs hosting and user accounts, and adds a second codebase. iCloud sync is already out of scope this term (FR-21).
Strict (closed) layering	Every ViewModel call to the data layer would need a domain method that only forwards it. That adds code with no behaviour behind it.
Calling the Gemini API straight from the app	The API key would have to ship inside the app, where it can be extracted and abused.

3 Quality attributes

Table 4 shows how the architecture meets each non-functional requirement in scope, and how we will check it.

Requirement	How the architecture addresses it	How we will check it
NFR-01 Performance: outfit visible within 2 s	The Today screen shows the cached outfit at once. Recommendation runs in an async task off the main thread. The remote call has a 1.5 s timeout and is skipped when offline.	Measure launch to outfit on an iPhone 12 with a 200-item test wardrobe, using Instruments.
NFR-02 Privacy: data stays on the device	All storage is local (SwiftData, sandbox files, UserDefaults). `RemoteAIClient` is the only component that sends wardrobe data, it sends attributes only, and `RemoteAIRecommender` checks consent before every call. The location goes to WeatherKit only after the user grants location permission.	Inspect network traffic with a proxy while consent is off: only weather requests should appear. Code review checks that no other component builds outgoing requests.
NFR-03 Availability: works offline	The rule-based recommender is always the last link and needs no network. `WeatherService` caches the last forecast, and the context is marked offline.	Run the app in airplane mode, then browse the wardrobe and generate an outfit.
NFR-04 Usability: first outfit within 5 minutes	Two tabs (Today, Wardrobe) and filters in a sheet. When the wardrobe is missing a top, a bottom or shoes, the Today screen shows an empty state with an Add item button instead of an error.	Time three to five people who have never seen the app.
NFR-05 Performance: under 1 s for 200 items	Per-slot shortlist before combining (section 2.3). The scorer is a pure Swift function with no input or output, so it can be timed in isolation.	XCTest performance test (`measure`) on 200 generated items.
NFR-06 Reliability: no data lost	SwiftData persistent store. Photos are saved before the item and removed if saving fails. Files live in Application Support, not in the caches folder that iOS may clear.	Force-quit during a save. Install a new build over an old one and check that all items remain.
NFR-07	Each adapter asks for its own permission the first time it is needed, with a purpose string in Info.plist. Every service returns a safe default when access is	Deny each permission in turn and check that the app still works.

Requirement	How the architecture addresses it	How we will check it
Security: permissions on first use	denied, and `ContextBuilder` handles missing data.	
NFR-08 Maintainability: swappable recommender	The `OutfitRecommender` protocol. TodayViewModel holds only the protocol, and the chain is assembled in one place, so adding or removing a recommender changes one line.	UI tests run with a fake recommender, with no change to any view.

Table 4. Non-functional requirements and the architectural decisions that meet them.

NFR-09 to NFR-11 are not formally tested this term, but the design supports them. Standard SwiftUI controls work with VoiceOver and Dynamic Type, and we will add labels to item photos. Strings go in a String Catalog so Arabic can be added later, and SwiftUI mirrors layouts for right-to-left languages. The deployment target is iOS 17, which SwiftData and `@Observable` need anyway.

4 Architectural and design patterns

Two architectural patterns shape the whole app (layers and MVVM). Five design patterns shape individual parts of it. For each one, the table gives what the pattern is, where we use it, and why it suits Quickdress.

4.1 Architectural patterns

Layered architecture

What it is	Components are grouped into layers by level of abstraction. Each layer uses only the layers below it.
Where it is used	Presentation, domain, and data and services (Figure 1). The layering is relaxed: a ViewModel may skip the domain layer when there is no logic in between.
Why it fits	It keeps Apple framework code at the bottom, where it can be replaced (for example WeatherKit by Open-Meteo) without touching the logic. The domain layer can be unit tested without a device. It also matches the order we build in: each sprint adds a vertical slice through all three layers.

Model, View, ViewModel (MVVM)

What it is	The View displays state, the ViewModel holds that state and the actions on it, and the Model is the data. The View observes the ViewModel and redraws when the state changes (an instance of the Observer pattern).
Where it is used	Every screen: TodayView with TodayViewModel, WardrobeView with WardrobeViewModel, and so on. ViewModels are `@Observable` classes.
Why it fits	SwiftUI is built around this kind of binding. ViewModels can be tested without any UI, and the people building screens and the people building logic can work in parallel.

4.2 Design patterns

Strategy

What it is	A family of interchangeable algorithms behind one interface, so the caller can use any of them without knowing which.
Where it is used	The `OutfitRecommender` protocol and its three implementations.
Why it fits	NFR-08 asks for this directly. We can ship the rule-based recommender in Sprint 2 and add Core ML or Gemini later without changing TodayViewModel.

Chain of Responsibility

What it is	A request is passed along a chain of handlers until one of them handles it.
Where it is used	CoreMLRecommender, then RemoteAIRecommender, then RuleBasedRecommender, linked through the `fallback` property. Each link is also a Strategy implementation, so the whole chain looks like one recommender to the caller.
Why it fits	Each recommender decides for itself whether it can answer, so the fallback logic is not one large if-else in the ViewModel. The order is set in one place, and the last link always answers (NFR-03).

Repository

What it is	An object that behaves like a collection of domain objects and hides how they are stored.
Where it is used	The `WardrobeRepository` protocol and `SwiftDataWardrobeRepository`.
Why it fits	All queries and the SwiftData `ModelContext` live in one class. Tests and SwiftUI previews use an in-memory fake. Storage could later move to iCloud (FR-21) without changes elsewhere.

Adapter

What it is	A class that wraps an existing interface so it matches the interface a client expects.
Where it is used	`WeatherKitService`, `EventKitCalendarService`, `CoreLocationProvider` and `FirebaseGeminiClient` wrap Apple and Google SDKs behind our own protocols and convert their types into our `Forecast` and `CalendarEvent`.
Why it fits	SDK types do not leak into the domain layer. If WeatherKit is not available (risk R1), we write one new adapter for Open-Meteo and nothing else changes.

Dependency injection

What it is	Objects receive the things they depend on from outside instead of creating them.
Where it is used	`QuickdressApp` creates the adapters, the recommender chain and the managers, and passes them to the ViewModels through their initialisers.
Why it fits	It is what lets tests and previews use fakes, and it turns the order of the recommender chain into a configuration decision rather than code spread across classes.

5 Detailed design: class diagram

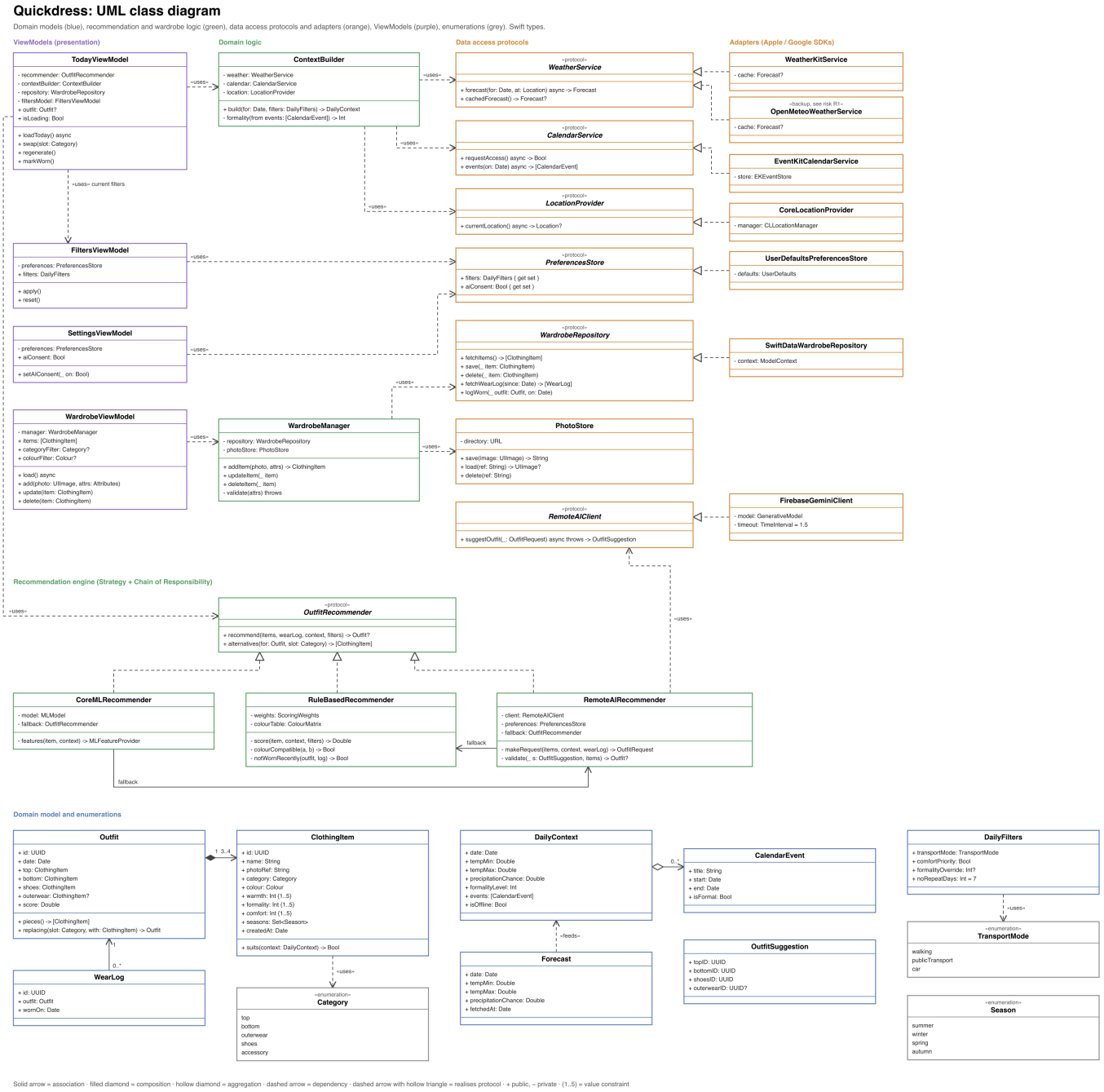


Figure 4. UML class diagram (Swift types). The components in Figure 1 are made up of these classes.

Table 5 maps each component in Figure 1 to its classes in Figure 4.

Component	Classes in Figure 4
Today screen	TodayViewModel
Wardrobe screens	WardrobeViewModel
Filters screen	FiltersViewModel
Settings screen	SettingsViewModel
WardrobeManager	WardrobeManager
ContextBuilder	ContextBuilder
Recommendation engine	OutfitRecommender, CoreMLRecommender, RemoteAIRecommender, RuleBasedRecommender
Domain model	ClothingItem, Outfit, WearLog, DailyContext, Forecast, CalendarEvent, DailyFilters, OutfitSuggestion, Category, Season, TransportMode
WardrobeRepository	WardrobeRepository, SwiftDataWardrobeRepository
PhotoStore	PhotoStore
WeatherService	WeatherService, WeatherKitService, OpenMeteoWeatherService (backup, see R1)
CalendarService	CalendarService, EventKitCalendarService
LocationProvider	LocationProvider, CoreLocationProvider
PreferencesStore	PreferencesStore, UserDefaultsPreferencesStore
RemoteAIClient	RemoteAIClient, FirebaseGeminiClient

Table 5. Mapping from components to classes.

Every protocol in the data layer has its adapter drawn next to it, joined by a realisation arrow. Dependencies that a class holds as a typed attribute, such as `TodayViewModel.repository`, are listed in the class box; only the main ones are drawn as arrows to keep the diagram readable.

6 Technical feasibility and risks

6.1 Feasibility

Every in-scope feature uses well-documented first-party Apple frameworks: SwiftUI, SwiftData, PhotosUI, WeatherKit, EventKit and Core Location. The only third-party SDK is Firebase, and only the optional Gemini step uses it. The hardest algorithm, the recommender, starts as rule-based scoring in ordinary Swift. Core ML and Gemini are optional links in the chain, and the app is complete without them.

The architecture also sets the build order. Each sprint delivers a working vertical slice through all three layers:

Sprint	Components built	Result
Sprint 1	Wardrobe screens, WardrobeManager, WardrobeRepository, PhotoStore, domain model	A working digital wardrobe (FR-01 to FR-04)
Sprint 2	Today screen, ContextBuilder, WeatherService, LocationProvider, RuleBasedRecommender, with CoreMLRecommender forwarding every request	A daily outfit matched to the weather (FR-05 to FR-08)
Sprint 3	CalendarService, Filters screen, PreferencesStore, wear log. If there is capacity: FR-12, FR-13 and FR-22 (Settings screen, RemoteAIRecommender, RemoteAIClient)	Calendar, filters and no repeats (FR-09 to FR-11)

Table 6. Build order by sprint.

6.2 Risks

ID	Risk	Likelihood	Impact	Mitigation
R1	WeatherKit needs a paid Apple Developer Program membership ($99 a year). Without one, the app also has to be reinstalled on test phones every 7 days.	High	High: blocks FR-08	Find out in week 1 whether the university or a team member has a membership. If not, write `OpenMeteoWeatherService` (free for non-commercial use, no key) behind the same `WeatherService` protocol.
R2	Xcode runs only on macOS. A team member without a Mac cannot build or run the app.	Medium	High	Check in week 1. Pair those members with Mac owners, or give them work that does not need Xcode: the colour compatibility table and scoring weights, test plans, Firebase setup and documentation.
R3	The team is learning Swift and SwiftUI during the project.	High	Medium	Sprint 1 is plain create, read, update and delete work. Every pull request is reviewed. The layers let each person work on one part at a time.
R4	There is no training data for the Core ML model.	High	Low	The rule-based recommender is the product. Core ML is only attempted if we can collect real usage data, which probably means after this term.
R5	Gemini replies may be wrong, slow or not valid JSON.	Medium	Low	Structured output, validation against the wardrobe, a 1.5 s timeout and the rule-based fallback.
R6	Gemini cost, quota or model names change.	Low	Low	Flash-Lite is Google's lowest-cost model in the family. The model name is in Remote Config. Usage is opt-in and limited to one call per generated outfit.
R7	Detecting formality from calendar titles by keyword is unreliable (vague titles such as "Meeting", or titles in Arabic).	High	Medium	The user can tag an event as formal. The formality override in the filters always wins. The keyword list is kept in one editable place.
R8	SwiftData is fairly new and has known bugs, especially with schema changes.	Medium	Medium	All access goes through `WardrobeRepository`, so we could move to Core Data. We keep the schema small and test upgrades (NFR-06).

ID	Risk	Likelihood	Impact	Mitigation
R9	Automatic category and colour suggestion (FR-13) may be inaccurate.	Medium	Low	It is a stretch goal, and the user always confirms or corrects the suggestion.
R10	The iOS Simulator has no camera.	Certain	Low	Use photo library import in the Simulator and test the camera on a real phone.

Table 7. Technical risks.

R1 and R2 need answers in the first week, because both could stop part of the team from building the app at all.

7 Conclusion

Quickdress is split into presentation, domain, and data and services layers, with MVVM on every screen. Each Apple framework and external service sits behind a protocol, and the recommender sits behind a single interface. Behind that interface is a fallback chain whose last link needs no network and sends no data, which is how the design meets the offline and privacy requirements while still allowing an on-device model and an opt-in Gemini step. Table 4 traces each of NFR-01 to NFR-08 to the decision that meets it, and Table 6 shows that the build order fits the three sprints in the Requirements Specification.

Part C

Presentation

Presentation outline

The milestone asks the presentation to cover a fixed list of topics. The deck itself is the separate file Quickdress*Milestone1*Presentation.pptx. Table C1 shows where each required topic is covered, the points to make, and where the material comes from.

#	Slide	Points to make	Source / figure
1	Title	Quickdress, team names, Milestone 1.	None
2	What it is and the problem	An iOS app that recommends one outfit a day from clothes you already own, matched to weather and calendar. Turns a 10-minute decision into a 10-second confirmation.	Part A, 1.1 and 1.2
3	Users and stakeholders	Students, young professionals, people with decision anxiety, people unsure about colour. Stakeholders: users, team, instructor and TAs, Apple, weather provider, Google (only for the opt-in AI step).	Part A, 1.3 and 1.4
4	Motivation	Closet apps like Cladwell, Whering and Acloset store clothes but still leave the choosing to the user. Quickdress adds day-aware filters and calendar integration.	Part A, 1.5
5	Product backlog	22 functional and 11 non-functional requirements, prioritised with MoSCoW.	Part A, section 2
6	Scope	FR-01 to FR-11 and NFR-01 to NFR-08 committed; FR-12, FR-13 and FR-22 are stretch goals; FR-14 to FR-21 deferred.	Part A, section 3
7	Why the scope is achievable	Three sprints, each ending in a demo. The riskiest work sits in the middle sprint, and the recommender starts as simple rules.	Part A, Figure 1 (sprint plan)
8	Use cases	One primary actor, three secondary actors, twelve use cases. Walk through "View today's outfit".	Part A, Figure 2 (use case diagram)
9	High-level architecture	Three layers with MVVM: presentation, domain, data and services. Every framework sits behind a protocol.	Part B, Figure 1 (component diagram)
10	How it works	The "View today's outfit" flow, then the fallback chain: Core ML, then opt-in Gemini, then rule-based, which always answers and needs no network.	Part B, Figures 2 and 3
11	Quality attributes and patterns	How the design meets performance, privacy, offline use and swappability. Patterns: Layered, MVVM, Strategy, Chain of Responsibility, Repository, Adapter, dependency injection.	Part B, sections 3 and 4
12	Technical risks	WeatherKit needs a paid developer account (backup: Open-Meteo); Xcode needs a Mac; the team is learning Swift; calendar keyword detection is unreliable.	Part B, Table 7
13	Global and social considerations	Privacy first (on-device data), weather-related health, modest-dress and cultural norms, re-wearing instead of buying, free app with no shopping links.	Part A, section 5

#	Slide	Points to make	Source / figure
14	Conclusion	What Milestone 1 fixes (baseline requirements and architecture) and what Sprint 1 delivers next (a working digital wardrobe).	Part A, section 6

Table C1. Presentation outline.